

CINEBANDIT

Adaptive Movie Recommendation System

Powered by Contextual Bandits · Built with MLOps

KEY PERFORMANCE METRICS

Metric	Value	Notes
Train CTR	68.3%	On training data
Test CTR	63.3%	On held-out data
Latency (p95)	< 25ms	Target: < 200ms
Arm Coverage	100%	All 1,682 movies

Chirag
DA25M008

DA5402 · MLOps

GitHub: github.com/chirag3211/cinebandit

1. Executive Summary

CineBandit is a production-grade adaptive movie recommendation system that learns from user feedback in real time. Built on the LinUCB (Linear Upper Confidence Bound) contextual bandit algorithm and deployed using a full MLOps stack, it addresses the core challenge faced by modern recommenders: balancing real-time personalisation against catalog-wide exploration.

Unlike traditional systems that rely on offline retraining cycles, CineBandit updates model parameters incrementally using the Sherman-Morrison formula ($O(d^2)$), enabling sub-25ms inference with continuous online adaptation.

Key Result: CTR improved from ~42% (random baseline) → 63.3% Test CTR (+21 percentage points)

1.1 Technology Stack

Component	Technology	Purpose
ML Algorithm	LinUCB	Contextual bandit
Dataset	MovieLens-100K	User ratings
API	FastAPI	Model serving
Frontend	Streamlit	User interface
Tracking	MLflow	Experiment management
Orchestration	Apache Airflow	Pipeline automation
Versioning	DVC	Data + model versioning
Monitoring	Prometheus + Grafana	Live metrics
Database	PostgreSQL	Backend persistence

2. Problem Statement & Objectives

Modern recommendation systems must simultaneously handle large catalog sizes, shifting user preferences, cold-start users, and strict latency requirements. Periodic offline retraining fails to capture rapid preference changes, while pure greedy approaches over-exploit known popular items and miss long-tail content.

2.1 Exploration vs. Exploitation

The core trade-off in any recommendation system is between exploitation (recommending items already known to work) and exploration (trying new items to learn user preferences). CineBandit resolves this using the Upper Confidence Bound (UCB) principle:

- Exploitation — recommend items with proven high expected reward
- Exploration — try under-tested items whose reward is uncertain
- UCB bonus — automatically widens the confidence bound for less-explored arms

2.2 Success Metrics

Metric	Target	Achieved	Status
Click-Through Rate (CTR)	> 0.55	0.633	✓ Exceeded
Inference Latency (p95)	< 200ms	< 25ms	✓ Exceeded
Arm Coverage	100%	100%	✓ Met
Learning Gain (Q1 → Q4)	> 5%	+10.9%	✓ Exceeded

3. System Architecture

CineBandit is deployed as a microservices architecture comprising 9 Docker containers, each with a distinct responsibility. The design separates concerns cleanly across three logical layers:

- Data & ML Layer — feature engineering, model training, parameter updates
- Serving Layer — FastAPI inference endpoint, request validation, response caching
- MLOps Layer — MLflow experiment tracking, Airflow DAG orchestration, Prometheus monitoring

3.1 Layer Summary

Layer	Components	Role
Data & ML	DVC pipelines, feature store	Data versioning and model logic
Serving	FastAPI + PostgreSQL	Low-latency inference
MLOps	MLflow, Airflow, Prometheus/Grafana	Full lifecycle management

4. ML Algorithm — LinUCB

LinUCB (Linear Upper Confidence Bound) is a contextual bandit algorithm that models expected reward as a linear function of the context vector. For each arm (movie), it maintains a regularised least-squares estimator and an uncertainty term. The arm with the highest upper confidence score is selected at each step.

4.1 Why LinUCB?

Feature	LinUCB	Collaborative Filtering	Deep RL
Online updates	Yes	No	Partial
Uncertainty-aware	Yes	No	No
Interpretability	High	Medium	Low
GPU required	No	No	Often
Cold-start handling	Yes	Poor	Poor

4.2 Update Rule

Parameters are updated incrementally using the Sherman-Morrison rank-1 update, avoiding costly matrix inversions on every feedback event. This yields $O(d^2)$ complexity per update — fast enough to run synchronously in the feedback handler.

Sherman-Morrison Update: $A^{-1} \leftarrow A^{-1} - (A^{-1}xx'A^{-1}) / (1 + x'A^{-1}x)$

4.3 Context Vector (d = 60)

Component	Dimensions	Description
User genre preferences	19	Normalised rating history per genre
Movie genre encoding	19	One-hot genre vector
Interaction features	19	Element-wise user × movie preferences
Metadata features	3	Popularity, recency, rating count

5. Data Pipeline

5.1 Dataset — MovieLens-100K

Total ratings: 100,000

Unique users: 943

Movies in catalog: 1,682 (full arm coverage)

Ratings are binarised (≥ 4 stars $\rightarrow$ positive reward) to frame the problem as a binary bandit. User and item features are extracted and standardised before being fed into the context vector.

5.2 DVC Pipeline Stages

Stage	Inputs	Outputs
prepare_data	Raw MovieLens files	Cleaned, split datasets
train_model	Training dataset	LinUCB parameters + MLflow run
evaluate	Test dataset + model	CTR metrics, drift scores

5.3 Airflow DAGs

DAG	Schedule	Purpose
data_ingestion	Daily	Load, validate, and version-stamp new data
drift_detection	Hourly	Monitor feature distribution shift via KL divergence
model_retraining	On trigger	Full LinUCB retraining when drift threshold exceeded

6. MLOps Implementation

6.1 MLflow Experiment Tracking

Every training run logs parameters (alpha, context dimension), metrics (train/test CTR, latency), and serialised model artifacts to MLflow. The Model Registry maintains versioned checkpoints with stage labels (Staging / Production), enabling zero-downtime promotion.

6.2 Drift Detection

The drift detection DAG computes KL divergence between the current rating distribution and a reference baseline. When divergence exceeds the configured threshold (default: 0.1), a model retraining run is automatically triggered, ensuring the system adapts to changing user behaviour.

KL Divergence threshold: 0.1 $\rightarrow$ automatic retraining triggered

6.3 Prometheus Monitoring

A custom FastAPI metrics exporter exposes the following gauges and histograms to Prometheus, visualised in a Grafana dashboard:

Metric	Type	Description
request_latency_seconds	Histogram	End-to-end inference latency
recommendation_ctr	Gauge	Rolling click-through rate
drift_score	Gauge	Current KL divergence score
model_errors_total	Counter	Cumulative error count

7. API Design (FastAPI)

Endpoint	Method	Purpose
/recommend	POST	Return ranked movie list for a user context
/feedback	POST	Record click/skip and update LinUCB parameters
/metrics	GET	Expose Prometheus-formatted metrics
/health	GET	Liveness probe for container orchestration

The /feedback endpoint performs the Sherman-Morrison update synchronously, ensuring model parameters reflect the latest user interaction within the same request cycle — a key differentiator from batch-retrain approaches.

8. Frontend (Streamlit)

The Streamlit dashboard provides three main views for interacting with and monitoring the system:

Screen	Features
Recommendations	Contextual movie suggestions, thumbs-up/down feedback buttons, real-time parameter update confirmation
Pipeline Console	Airflow DAG run history, MLflow experiment browser, model registry status
Monitoring Dashboard	Live Prometheus charts (CTR, latency, drift score), error rate timeline

9. Testing

The project maintains a comprehensive test suite covering unit, integration, and end-to-end scenarios.

Total tests: 102
Pass rate: 100% (all suites green)

Test coverage spans the LinUCB update logic, feature extraction pipeline, API endpoint contracts, Airflow DAG integrity, and MLflow artifact consistency.

10. Results & Performance

10.1 CTR Comparison

Model / Condition	CTR	Delta vs Baseline
Random baseline	~0.42	—
CineBandit (train)	0.683	+26.3 pp
CineBandit (test)	0.633	+21.3 pp

10.2 Temporal Learning Gain

CTR measured across four equal chronological segments of the test set shows consistent improvement as the model accumulates feedback:

Quarter	Approximate CTR	Change
Q1 (earliest)	~0.57	Baseline period
Q2	~0.60	+3 pp
Q3	~0.62	+5 pp
Q4 (latest)	~0.633	+10.9 pp total

Key Insight: The model demonstrably improves over time — confirming genuine online learning rather than static lookup.

11. Challenges & Resolutions

Challenge	Root Cause	Resolution
Airflow permission errors	UID mismatch in container	Fixed file ownership in entrypoint script
MLflow artifact serving	Proxy path misconfiguration	Served artifacts via reverse proxy with correct base URL
Prometheus scrape errors	Metrics endpoint not mounted	Fixed exporter registration and route binding

12. Conclusion

CineBandit demonstrates that a contextual bandit approach, when combined with a thoughtfully engineered MLOps stack, can deliver a fully adaptive recommendation system with production-grade reliability. The system achieves a 63.3% test CTR — 21 percentage points above a random baseline — while maintaining sub-25ms p95 latency and 100% catalog coverage.

The project validates three core contributions:

- Real-time learning via Sherman-Morrison incremental updates eliminates the need for offline retraining cycles.
- Full MLOps pipeline (DVC, MLflow, Airflow, Prometheus) ensures reproducibility, auditability, and automated drift response.
- Strong empirical performance (+10.9% CTR gain over the test period) confirms genuine online adaptation.

Future Work

Direction	Expected Benefit
Scale to larger datasets (MovieLens-1M, 10M)	Validate performance at production scale
Neural contextual bandits (NeuralUCB)	Capture non-linear preference patterns
Real user deployment with A/B testing	Ground-truth CTR measurement
Kubernetes / Helm chart packaging	Cloud-native horizontal scaling

13. AI Assistance Disclosure

In the interest of academic and professional transparency, this section documents the use of AI tools (specifically Claude by Anthropic) during the development and documentation of the

CineBandit project. AI assistance was used as a learning and productivity aid; all design decisions, implementation, and validation were carried out by the author.

Areas Where AI Assistance Was Used

Area	Description
Grafana Panel Configuration	Sought guidance on structuring Prometheus-backed Grafana panels — including query syntax, panel types, and dashboard layout best practices for monitoring CTR, latency, and drift metrics.
LinUCB Algorithm Internals	Used AI to deepen understanding of the LinUCB algorithm — specifically the UCB score derivation, the role of the exploration parameter alpha, and the mathematical intuition behind the Sherman-Morrison incremental update.
MLflow Debugging	Consulted AI assistance to diagnose and resolve MLflow-related issues, including artifact path misconfiguration, model registry promotion errors, and experiment run logging inconsistencies.
Unit Test Generation	AI was used to generate a comprehensive suite of unit tests covering the LinUCB update logic, feature extraction pipeline, API endpoint contracts, and Airflow DAG integrity — totalling 102 passing tests.
verification.md	A verification.md file was created with AI assistance to define a structured manual and automated testing checklist, ensuring end-to-end system correctness across all microservices and pipeline stages.

All AI-assisted content was reviewed, tested, and validated by the author. The use of AI tools is consistent with the project's emphasis on modern engineering practices.

github.com/chirag3211/cinebandit